

구조 기반 산출물 생성 스킬

산출물 생성 스킬이 없을 때, spec-driven-generation이 기본





오늘 알게 될 3가지



🔄 Cocrates가 사용자의 요청을 받으면 어떻게 어떤 스킬을 쓸지 라우팅하는가



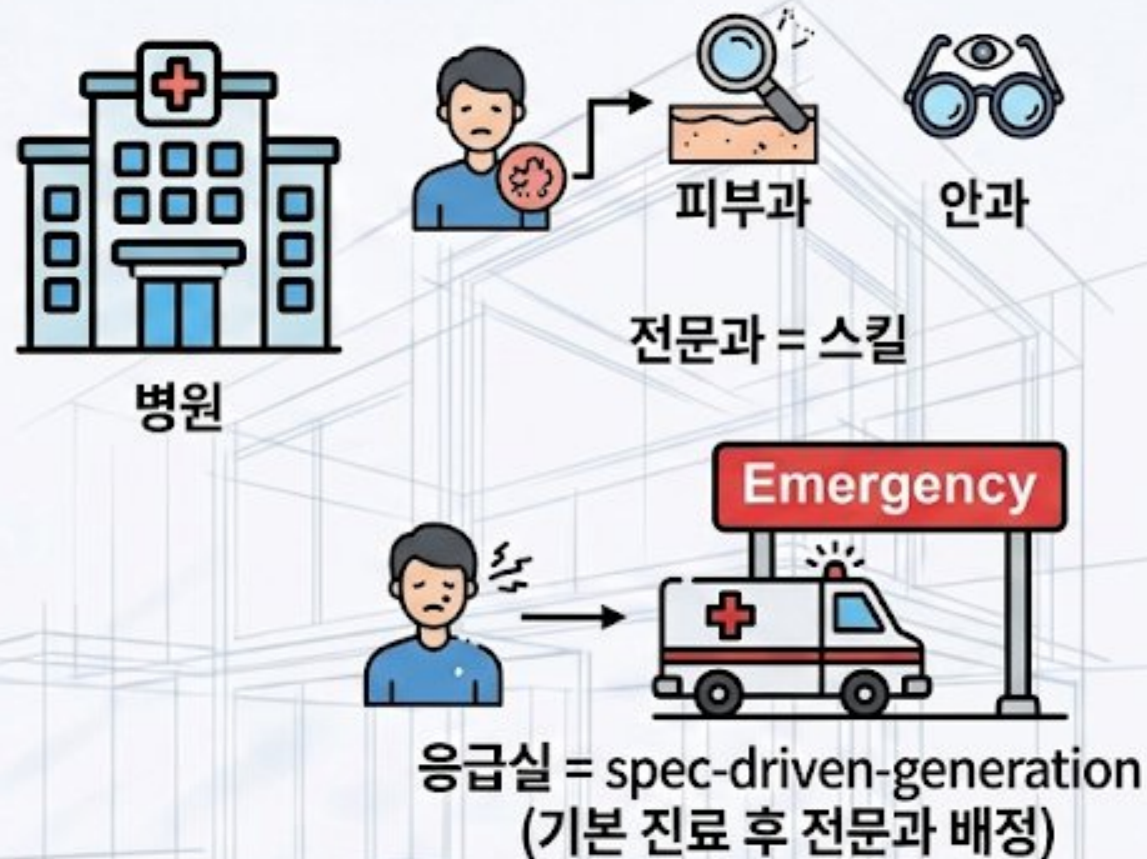
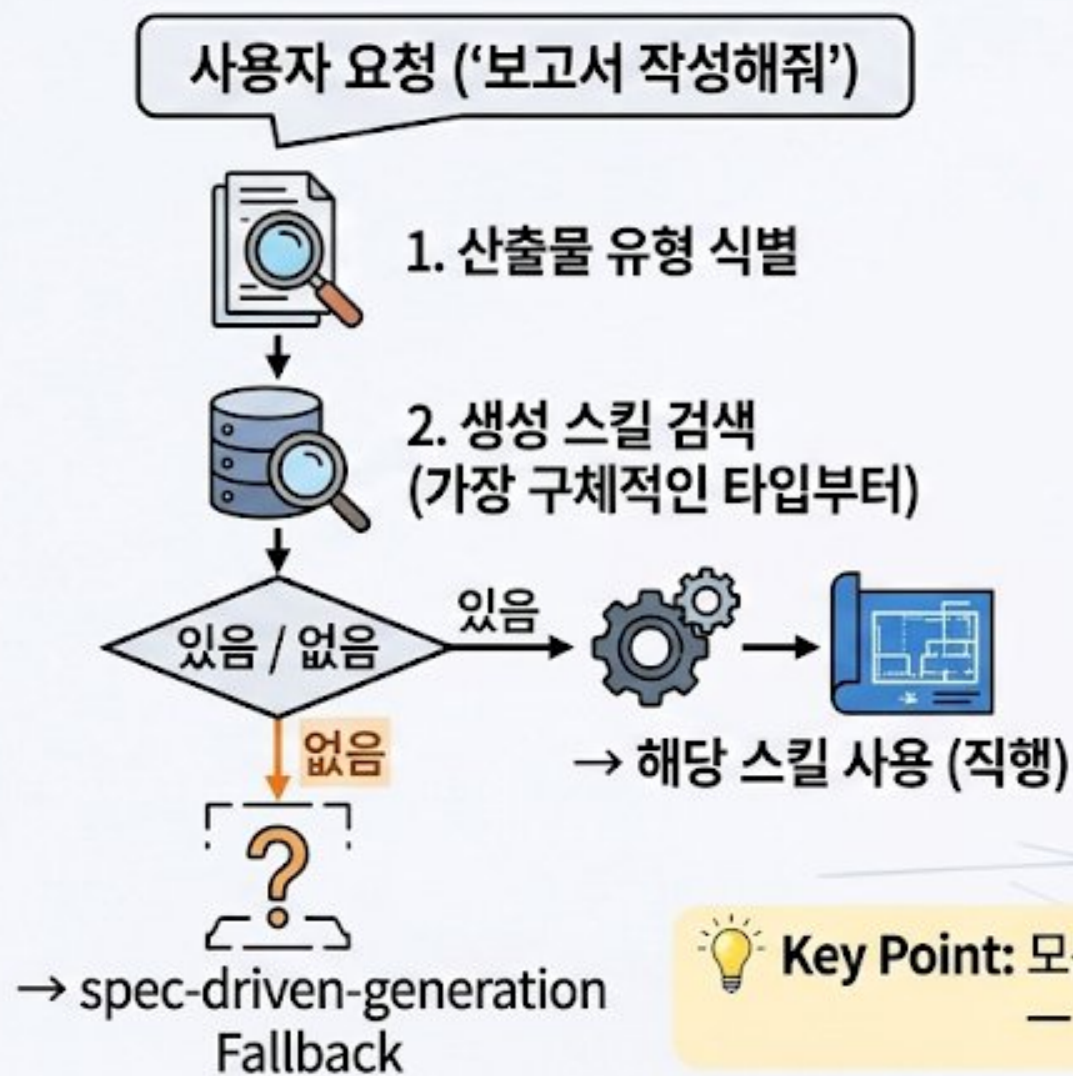
🚦 핵심 스킬 중 하나인 spec-driven-generation이 spec이 없을 때 어떻게 spec 작성을 유도하는가



🔧 4개 스킬 각각의 핵심 원칙과 절대 금지 사항



↻ Intent-to-Skill Routing — 요청이 스킬이 되는 순간



Key Point: 모든 요청이 spec-driven-generation을 거치는 것은 아니다.
— 스킬이 없을 때만 fallback.



spec-driven-generation Fallback — “spec이 없으면 spec을 만든다”

역할 1: Spec Readiness Gate — Spec 준비 확인



Spec이 준비되었는지
확인합니다.

역할 2: 생성 — 준비된 Spec 기준으로 산출물 생성



준비된 Spec
기준으로 산출물을
만듭니다.



Key Point: “생성하기 전에 멈출 줄 안다”
— 이것이 spec-driven-generation의 위대한 점입니다.



adr-writing — “왜 이런 결정을 내렸는가”



Architecture Decision Record (ADR)

핵심 원칙: 1 ADR = 1 Concern

• 하나의 ADR 파일 = 하나의 관심사만 다룸

적용 범위:

- 기술, 문서 구조
- 발표 전개 방식
- 모든 구조적 결정

✗ 가짜 대안 제시 금지

- 모든 대안은 실질적이어야 함.
- 장단점이 팽팽해야 함.



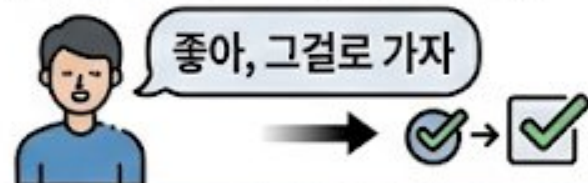
✗ 산문 스타일 금지

- 대안 이름 + 2~4개 불릿으로 요약.
- 장문(예: ‘저는 생각합니다...’) 안 됨.



✗ 무단 승인 금지

- AI 혼자 ‘approved’ 설정 금지.
- 사용자 명시적 구두 승인 필수 (proposed → approved).



예시: 보고서 서론 구조 — chronological vs problem-first vs executive summary (장단점 2~3줄 요약)



Key Point: ADR은 결정을 강요하는 것이 아니라, 결정을 가능하게 하는 장치입니다.



spec-writing — “무엇을 만들기로 했는가”



핵심 원칙:
자체 완결성
(Self-Containment)
— 이 문서 하나면 OK

ADR 참조 금지

- "adr/01.md 참고" 금지.
결과만 명시.

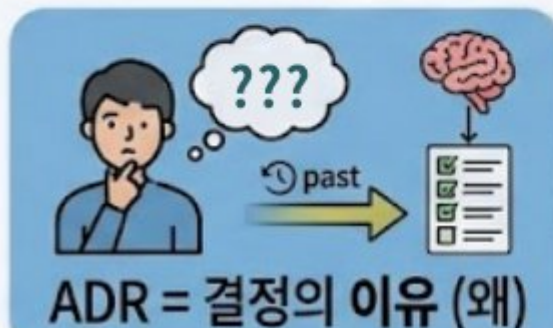


장황한 줄글 금지

- 1 불릿 = 1 검증 가능 요구사항.
구체적 표현 필수.



Spec vs ADR 비교



Spec의 세 가지 역할

-  생성의 유일한 기준점
-  검증의 유일한 기준점
-  의사소통의 공통 참조점



"이 Spec대로 만들었습니다" — 이 한마디면 모든 논란이 끝납니다.



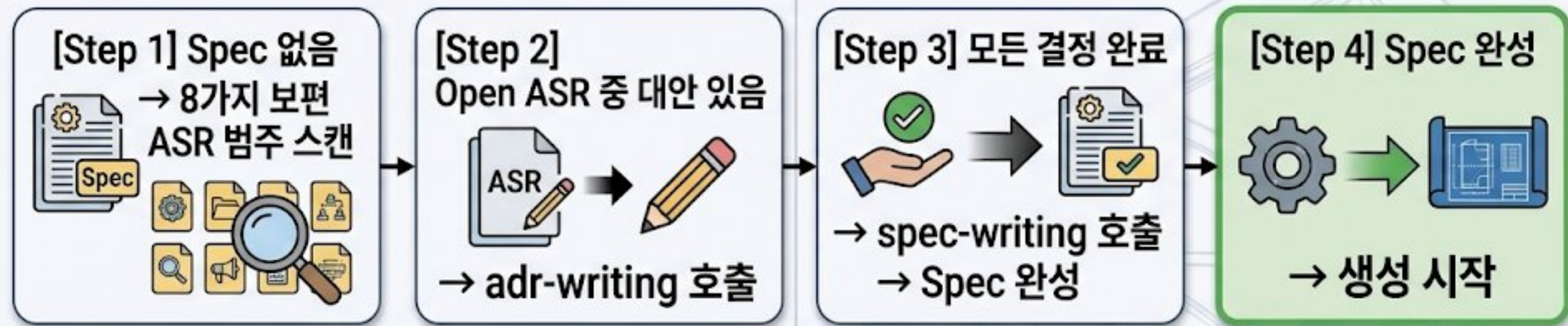
Spec은 ADR(과거 기록)과 Generation(미래 결과물) 사이의 현재 설계도입니다.



⚙️ spec-driven-generation – “설계도대로 짓는다”

Spec이 유일한 헌법이자 근거

ASR Readiness Gate



일단 벽돌부터
쌓아볼까요?

Spec 우선 원칙:
최초 프롬프트보다
Spec을 우선시

잠깐, 설계도부터
확인합시다.



spec-driven-verification — “설계도와 대조하라”

검증 핵심 원칙: 이중 목적 검증 — Deviation(이탈) + Undocumented ASR(미등록 ASR)

1. Deviation (이탈) - Spec 명시와 다른 구현

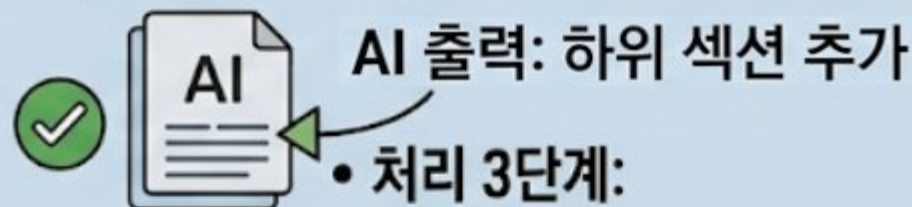


- 예시: PostgreSQL 쓰라니까 MongoDB
- 원인: 설계 실수, 구현 충돌, AI 판단
- 사용자 재검토 (어느 쪽이 잘못됐는지 판단)

숨은 목적: ASR 놓친 경험
→ 사용자 역량 개선

2. Undocumented ASR (미등록 ASR)

- Spec에 없었지만 AI가 구현한 내용



1 단계: ✖
무시 (Ignore)
— 중요하지 않음
→ Spec 미포함

2 단계: 👍
수용 (Accept)
— 중요하고 타당
→ Spec에 포함

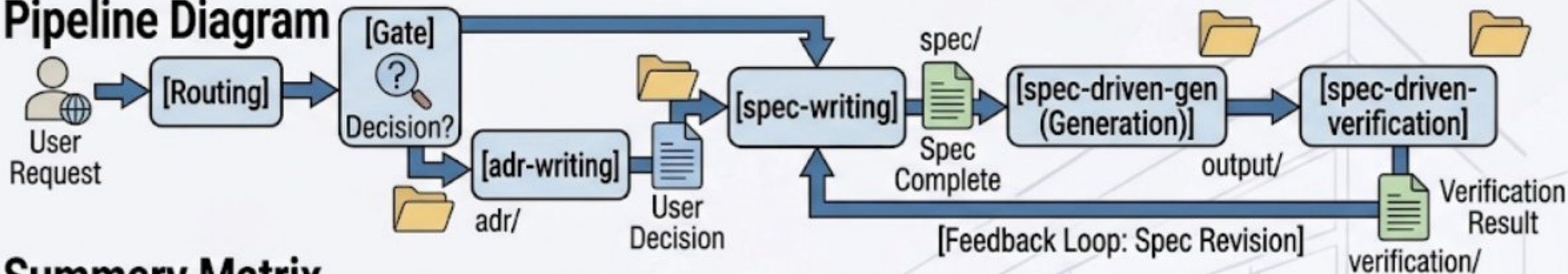
3 단계: 🔍
ADR 검토 (Escalate)
— 중요하지만 판단 어려움
→ ADR로

💡 검증은 생성물 검사 + 사용자의
다음 설계를 개선하는 피드백 루프



🔗 파이프라인 연결 + 요약 매트릭스

Pipeline Diagram



Summary Matrix

	Skill (스킬)	Core Role (핵심 역할)	Never Allowed (절대 금지 사항)
	[adr-writing]	Deconstruction of Alternatives & Decision Recording	Fake Alternatives, Prose Style, Unauthorized Approval ❌
	[spec-writing]	Integrated Decision Blueprint	ADR Reference, Long-form Prose ❌
	[spec-driven-gen]	Generation based on Spec	Generation without Spec (Gate must be passed) ❌
	[spec-driven-ver]	Discovery of Deviation + Undoc ASR	Arbitrary Modification without User Confirmation ❌

💡 Each skill focuses on its own role, **communicating only through interfaces (files)**



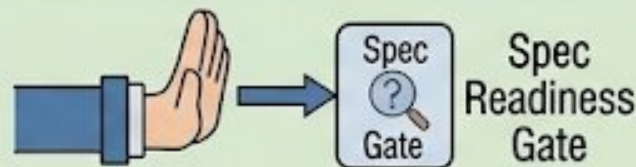
💡 핵심 통찰 + 학습 확인

3가지 핵심 통찰

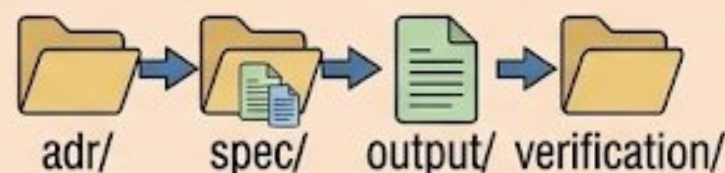
🔄 “스킬이 없으면 spec-driven-generation이 폴백된다”



🚦 “생성하기 전에 멈출 줄 안다”
— Spec Readiness Gate



🔗 “각 스킬은 파일로만 소통한다”
— 명확한 인터페이스



학습 확인

? 사용자가 “보고서 작성해줘”
→ Cocrates의 동작 순서는?

? spec-driven-generation,
spec 없을 때 3단계는?

? adr-writing의 3가지 금지 규칙은?

? Undocumented ASR 발견 시
3가지 처리 방법은?



🔮 다음 에피소드 + 마무리

 **Ep10. 원리 터득**
Generation 파이프라인 원리 학습

User Request → [Routing] → skill? → spec-driven-gen
[X No] Fallback

Generation 파이프라인 원리 학습 ✓

 **Ep11. 메커니즘 정복**
4개 전담 스킬 메커니즘 학습

adr-writing spec-driven-generation
spec-writing spec-driven-verification

4개 전담 스킬 메커니즘 학습 ✓

 **Ep12. 메타-스킬**
스킬 생성 메칭

A  Generating → Meta-Skill

스킬 생성 메타-스킬
(generating-skill-creation) 학습 예정 ➡

... “구조를 지배하는 자가 결과물의 주권을 집니다.
그리고 최고의 구조는 다시 자동화된 스킬로 박제됩니다.”
구조적 접근법 자체를 스스로 스킬화하는 마법 - 기대해 주세요.

1 ✓ ADR-Writing 스킬 완료 

2 ✓ Spec-Writing 스킬 완료 

3 ✓ Spec-Driven-Generation 스킬 완료 

4 ✓ Spec-Driven-Verification 스킬 완료 

Episode 11 마침.

